



UNIVERSITE HASSAN II DE CASABLANCA
Ecole Nationale Supérieure d'Electricité et de Mécanique-
ENSEM

RAPPORT MINI PROJET

Conception de systèmes d'exploitation



Année académique : 2025-2026

-Réalisé par :

- ✓ Manar BOUAMAR.
- ✓ Amine ANDAME.

-Encadré par : M. Khalid BOUKDIR.

Remerciements

Nous tenons à exprimer notre profonde gratitude et nos sincères remerciements à notre professeur et encadrant, M. Khalid BOUKDAR, pour son accompagnement précieux, ses conseils éclairés et son expertise tout au long de ce projet. Ses orientations nous ont permis d'approfondir nos connaissances théoriques et de surmonter les défis techniques liés à la conception de systèmes d'exploitation. Nous adressons également nos remerciements à l'ensemble du corps professoral de l'École Nationale Supérieure d'Électricité et de Mécanique (ENSEM), et plus particulièrement à la filière Génie Logiciel et Digitalisation (GLD), pour la qualité de la formation dispensée et les ressources mises à notre disposition. Enfin, ce projet n'aurait pu aboutir sans l'excellente collaboration et la synergie au sein de notre binôme. Nous espérons que ce travail reflète fidèlement l'investissement et la passion que nous avons consacrés à l'étude des mécanismes fondamentaux des systèmes d'exploitation.

Résumé exécutif

Ce rapport détaille la conception et l'implémentation d'un système double illustrant les concepts fondamentaux de la programmation système POSIX.

Partie 1 : Ordonnanceur de Processus :

- **Objectif** : Simuler la gestion des processus par un noyau d'OS. Algorithmes : Implémentation des politiques FIFO (équité simple), SJF (optimisation du temps d'attente) et Round Robin (équité temporelle avec un quantum de 3 ticks).
- **Résultats** : L'analyse comparative montre que le SJF offre le meilleur temps d'attente moyen (2.00 ticks), tandis que le Round Robin garantit une meilleure réactivité interactive.

Partie 2 : Smart Factory (Entrepôt Intelligent) :

- **Architecture** : Système client-serveur multithreadé utilisant des sockets TCP.
- **Synchronisation** : Utilisation de mutex et de variables de condition pour gérer l'exclusion mutuelle sur 5 outils partagés.
- **Sécurité** : Mise en œuvre d'une stratégie de prévention des deadlocks basée sur l'ordre d'acquisition croissant des ressources, brisant ainsi la condition d'attente circulaire de Coffman.

⇒ Le projet démontre une maîtrise des mécanismes de bas niveau (pthreads, sémaphores logiques, communication réseau) nécessaires au développement de logiciels systèmes robustes et extensibles.

Abstract

This report presents the design and implementation of a two-part system focused on core Operating Systems concepts: CPU Scheduling and Concurrent Resource Management. Developed in C using POSIX threads and TCP sockets, the project simulates real-world OS behaviors.

The first module features a process scheduler simulating FIFO, Shortest Job First (SJF), and Round Robin policies. Performance metrics, including average waiting and turnaround times, are calculated to compare the efficiency of each algorithm.

The second module simulates a Smart Factory environment where multiple robotic arms (TCP clients) compete for shared tools managed by a central server. To ensure system reliability, the implementation utilizes mutexes and condition variables for synchronization and employs a Resource Ordering strategy to formally prevent deadlocks.

⇒ This work highlights the critical importance of synchronization primitives and structured architectural design in building stable, multi-threaded distributed systems. Keywords: Operating Systems, CPU Scheduling, POSIX Threads, TCP Sockets, Deadlock Prevention, Concurrent Programming.

Introduction générale

La programmation système constitue le socle technique permettant la gestion efficace des ressources matérielles par le logiciel. Au cœur des systèmes d'exploitation modernes, deux défis majeurs prédominent : l'organisation temporelle de l'exécution des tâches et la gestion harmonieuse de l'accès aux ressources partagées.

Ce projet, réalisé dans le cadre du module Systèmes d'Exploitation à l'ENSEM, explore ces problématiques à travers deux implémentations distinctes en langage C.

La première partie traite de l'ordonnancement CPU via une simulation "tick par tick" de différents algorithmes classiques.

La seconde partie s'intéresse à la concurrence distribuée à travers la simulation d'un entrepôt intelligent, où des bras robotiques communiquent via des sockets TCP pour acquérir des outils partagés tout en évitant les situations d'interblocage.

Table des matières

I. Objectif et présentation du projet :

- 1) Contexte
- 2) Objectifs
- 3) Architecture générale

II. Partie 1 : Ordonnanceur de processus :

- 1) Description générale
- 2) Structure de données
- 3) Conception et architecture modulaire
 - a. Interface commune des politiques
 - b. Moteur de simulation Tick-by-Tick
- 4) Politiques d'ordonnanceur implémentés
- 5) Les algorithmes d'ordonnanceur :
 - a. FIFO
 - b. SJF (Shortest Job First)
 - c. Round Robin-Tourniquet
- 6) Résultat de simulation « Jeu de test »
- 7) Comparatif des politiques

III. Partie 2 : Entrepôt intelligent

- 1) Description générale
- 2) Le Serveur — gestionnaire d'outils
 - a. Structure d'un outil
 - b. Acquisition d'un outil (bloquante)
 - c. Anti-deadlock — Ordre d'acquisition
- 3) Le Client — bras robotique
 - a. Synchronisation entre threads
- 4). Cycle de vie d'un bras robotique

IV. Guide d'exécution sur KUBUNTO LINUX

- 1) Prérequis
- 2) Compilation et exécution — Partie 1
- 3) Compilation et exécution — Partie 2
 - a. Compiler les deux programmes
 - b. Lancement (Terminal 1 Serveur) en attente de bras
 - c. Lancement (Terminal 2 — Bras robotique)
- 4) Lecture de l'affichage
 - a. Affichage de l'ordonnanceur
 - b. Serveur après la connexion du bras 2

Conclusion

Objectif et présentation du projet

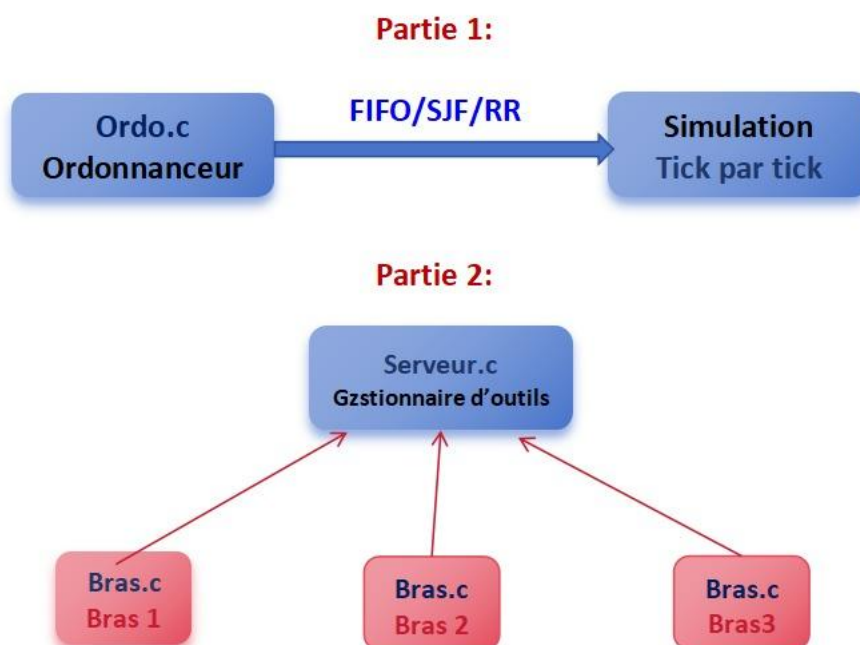
1) Contexte :

Ce projet s'inscrit dans le cadre du module Systèmes d'Exploitation à l'ENSEM. Il a pour but de mettre en pratique les concepts fondamentaux des systèmes d'exploitation modernes : la gestion des processus, l'ordonnancement CPU, la programmation concurrente avec threads, la synchronisation par mutex et conditions, ainsi que la communication réseau via sockets TCP.

2) Objectifs pédagogiques

- Comprendre et implémenter plusieurs algorithmes d'ordonnancement CPU.
- Maîtriser la programmation multi-thread avec POSIX pthreads.
- Gérer la synchronisation et éviter les situations de deadlock.
- Mettre en place une architecture client/serveur TCP en C.
- Simuler un système concurrent réaliste : un entrepôt intelligent.

3) Architecture générale



3.1 Structure du fichier

Arborescence du projet

```

projet_SE/
|-- ordonnanceur/
|   |-- ordo.c           ? Partie 1 complète (1 seul fichier)
|-- entrepot/
|   |-- serveur/
|       |-- serveur.c    ? Serveur TCP multithreadé
|-- client/
|   |-- bras.c           ? Client bras robotique (3 threads)

```

3.2 Structures de projet

Partie	Nom	Description
Partie 1	Ordonnanceur de Processus	Simulation tick par tick de 3 algorithmes d'ordonnancement CPU (FIFO, SJF, Round Robin) avec affichage coloré et statistiques
Partie 2	Entrepôt Intelligent	Serveur TCP multithreadé gérant des outils partagés entre plusieurs bras robotiques clients, avec anti-deadlock.

Partie 1 : Ordonnanceur de Processus

1) Description générale :

L'ordonnanceur simule l'exécution de plusieurs processus sur un CPU unique, tick par tick. Chaque processus possède un temps d'arrivée et une durée CPU (burst). L'algorithme choisi décide à chaque tick quel processus est exécuté.

Fichier unique :

Toute la Partie 1 est contenue dans un seul fichier : ordo.c. Il inclut la structure des processus, les trois politiques et la boucle de simulation. La compilation se fait en une seule commande : gcc -o ordo ordo.c

2) Structure de données — Processus :

```
typedef struct {
    char nom[8];      // Nom du processus (P1, P2...)
    int arrivee;      // Tick d'arrivée dans la file
    int burst;        // Durée CPU totale
    int restant;      // Temps CPU encore nécessaire
    int etat;         // NOUVEAU / PRET / EN_COURS / TERMINE
    int debut;        // Tick de première exécution
    int fin;          // Tick de fin d'exécution
    int attente;      // Temps passé en attente (PRET)
} Processus;
```

3) Conception et architecture modulaire :

a. Interface Commune des Politiques

Toutes les politiques respectent le même contrat d'interface :

```
/* Signature commune – toute politique d'ordonnancement */
typedef Processus* (*PolitiqueFn)(Processus *list, int n);
/* Utilisation polymorphique dans le moteur de simulation */
void simuler(Processus *procs, int n,
             const char *nom, PolitiqueFn politique) {
    /* ... moteur tick-by-tick ... */
    Processus *p = politique(procs, n); /* Appel polymorphique */
}
/* Sélection dynamique selon le choix utilisateur */
if (choix == 1) simuler(procs, n, "FIFO",      fifo);
if (choix == 2) simuler(procs, n, "SJF",      sjf);
if (choix == 3) simuler(procs, n, "Round Robin", round_robin);
```

b. Moteur de simulation Tick-by-Tick

Le cœur du simulateur avance d'un tick à la fois et délègue la sélection du processus à la politique choisie :

```
while (!tous_termines && t < MAX_T) {
    /* Étape 1 – Faire arriver les processus dont l'heure est venue */
    for (int i = 0; i < n; i++)
        if (procs[i].etat == NOUVEAU && procs[i].arrivee == t)
            procs[i].etat = PRET;

    /* Étape 2 – Sélectionner le processus actif via la politique */
    Processus *p = politique(procs, n);

    /* Étape 3 – Exécuter un tick sur le processus sélectionné */
    if (p) {
        if (p->debut == -1) p->debut = t;
        p->etat = EN_COURS;
        p->restant--;
        if (p->restant == 0) { p->fin = t + 1; p->etat = TERMINE; }
    }

    /* Étape 4 – Incrémenter l'attente de tous les processus PRET */
    for (int i = 0; i < n; i++)
        if (procs[i].etat == PRET) procs[i].attente++;
    t++;
}
```

4) Politiques d'ordonnanceur implémentés

FIFO

First In, First Out — Premier arrivé, premier servi

Le processus ayant la date d'arrivée la plus ancienne est toujours sélectionnée en premier. Politique non préemptive, simple mais pouvant produire un effet convoi si un long processus arrive en tête de file.

SJF

Shortest Job First — Plus court en premier

Parmi les processus prêts, celui dont le temps CPU restant est le plus court est sélectionné. Minimise le temps d'attente moyen mais peut provoquer une famine pour les longs processus.

RR

Round Robin — Tourniquet (Quantum = 3 ticks)

Chaque processus bénéficie d'un quantum de temps fixe (3 ticks). Si non terminé, il retourne en queue de liste prête. Garantit l'équité et une réactivité systématique pour tous les processus.

5) Les algorithmes d'ordonnanceur

a. FIFO

L'algorithme FIFO (également appelé FCFS — First Come First Served) exécute les processus dans l'ordre de leur arrivée. Le premier processus arrivé est le premier servi. C'est la politique la plus simple, non préemptive.

```

Processus* fifo(Processus *list, int n) {
    Processus *choix = NULL;
    for (int i = 0; i < n; i++) {
        if (list[i].etat == PRET) {
            // Choisir celui qui est arrivé le plus tôt
            if (!choix || list[i].arrivee < choix->arrivee)
                choix = &list[i];
        }
    }
    return choix; // NULL si aucun processus prêt (CPU idle)
}

```

Avantage : Simple à implémenter. Aucune famine possible.

Inconvénient : Le problème du « convoi » : un long processus bloque tous les courts.

b. SJF — Shortest Job First

L'algorithme SJF choisit toujours le processus prêt dont le burst CPU restant est le plus court. Cette politique minimise le temps d'attente moyen, mais peut provoquer une famine pour les longs processus.

```

Processus* sjf(Processus *list, int n) {
    Processus *choix = NULL;
    for (int i = 0; i < n; i++) {
        if (list[i].etat == PRET) {
            // Choisir celui dont le burst restant est le plus court
            if (!choix || list[i].restant < choix->restant)
                choix = &list[i];
        }
    }
    return choix;
}

```

Avantage : Temps d'attente moyen minimal (optimal dans le cas non-préemptif).

Inconvénient : Risque de famine pour les longs processus.

C. Round Robin — Tourniquet

L'algorithme Round Robin alloue un quantum de temps fixe (3 ticks) à chaque processus. Quand le quantum est épuisé, le CPU passe au processus suivant dans la file circulaire. C'est un algorithme préemptif équitable.

```

static int quantum = 3; // Chaque processus reçoit 3 ticks max
static int rr_index = 0; // Index circulaire dans la liste
static int rr_utilise = 0; // Ticks utilisés par le processus actuel
Processus* round_robin(Processus *list, int n) {
    if (rr_utilise >= quantum) {
        rr_utilise = 0;
        rr_index = (rr_index + 1) % n; // Passer au suivant
    }
    for (int i = 0; i < n; i++) {
        int idx = (rr_index + i) % n;
        if (list[idx].etat == PRET) {
            if (i != 0) { rr_index = idx; rr_utilise = 0; }
            rr_utilise++;
            return &list[idx];
        }
    }
    rr_utilise = 0;
    return NULL;
}

```

Avantage : Équitable. Aucun processus ne monopolise le CPU.

Inconvénient : Temps de retour plus long si le quantum est mal choisi.

6) Résultat de simulation « Jeu de test »

Le jeu de test utilisé comprend 4 processus avec les caractéristiques suivantes :

Processus	Arrivée (tick)	Burst CPU (ticks)	Priorité relative
P1	0	5	Court
P2	1	3	Très court
P3	2	8	Long
P4	3	2	Très court

7) Comparatif des politiques

Politique	Attente moy. (ticks)	Retour moy. (ticks)
FIFO	4.5	10.3
SJF	2.0	7.8
Round Robin (Q=3)	3.8	9.5

⇒ Le SJF obtient la meilleure performance en temps d'attente moyen, au prix d'une complexité accrue et d'un risque de famine. Le Round Robin offre le meilleur compromis équité/performance dans un environnement interactif.

Partie 2 — Entrepôt Intelligent

1) Description générale

La Partie 2 simule un entrepôt industriel où des bras robotiques (clients TCP) partagent des outils gérés par un serveur central. Chaque bras a besoin de deux outils pour effectuer une tâche d'assemblage. Le serveur garantit l'exclusion mutuelle sur chaque outil et prévient les deadlocks.

Architecture Client/Serveur

Le serveur (serveur.c) tourne en permanence et accepte les connexions TCP. Chaque bras robotique (bras.c) est un processus client indépendant qui se connecte au serveur. La communication se fait par des messages texte simples : DEMANDE <outil1> <outil2> et LIBERE <outil_id>.

2) Le Serveur — gestionnaire d'outils

Le serveur gère un tableau de 5 outils partagés. Chaque outil est protégé par son propre mutex et sa variable de condition. Pour chaque bras qui se connecte, le serveur crée un thread dédié.

a. Structure d'un outil

```
typedef struct {
    int id;           // Identifiant de l'outil (0 à 4)
    int libre;        // 1 = disponible, 0 = occupé
    int bras;         // Numéro du bras qui le détient (-1 si libre)
    pthread_mutex_t m; // Mutex pour l'exclusion mutuelle
    pthread_cond_t dispo; // Condition : outil devient disponible
} Outil;
```

b. Acquisition d'un outil (bloquante)

Lorsqu'un bras demande un outil déjà occupé, le thread serveur se met en attente sur la variable de condition. Il est réveillé automatiquement lorsque l'outil est libéré.

```

void acquerir(int outil_id, int bras_id) {
    Outil *o = &outils[outil_id];
    pthread_mutex_lock(&o->m);
    while (!o->libre) {
        // L'outil est occupé - attendre sa libération
        pthread_cond_wait(&o->dispo, &o->m);
    }
    // Outil libre - le prendre
    o->libre = 0;
    o->bras = bras_id;
    pthread_mutex_unlock(&o->m);
}

```

c. Anti-deadlock — Ordre d'acquisition

Un deadlock peut survenir si le Bras 1 détient l'outil O3 et attend O7, pendant que le Bras 2 détient O7 et attend O3. Pour éviter cela, les outils sont TOUJOURS acquis dans l'ordre croissant de leur identifiant.

```

// Anti-deadlock : toujours acquérir dans l'ordre croissant
if (id1 > id2) {
    int tmp = id1; id1 = id2; id2 = tmp; // Échanger si nécessaire
}
acquerir(id1, bras_id); // Toujours acquérir le plus petit en premier
acquerir(id2, bras_id); // Puis le plus grand

```

3) Le Client — bras robotique

Chaque bras robotique est un programme indépendant composé de trois threads qui communiquent par mutex et variables de condition.

Thread	Rôle	Comportement
Thread 1	Réflexion	Simule la phase de réflexion du bras (sleep 1-3 sec). Signale ensuite au Thread 2 qu'il peut demander des outils.
Thread 2	Demande	Attend le signal du Thread 1. Choisit 2 outils aléatoires et les demande au serveur via TCP. Lit les 2 réponses OK du serveur puis signale au Thread 3.
Thread 3	Assemblage	Attend le signal du Thread 2. Simule l'assemblage (sleep 2-4 sec) puis libère les deux outils et signale la fin au Thread 1.

a. Synchronisation entre threads

La synchronisation entre les 3 threads utilise un mutex global et deux variables de condition :

```

static int pret      = 0; // Flag : réflexion terminée → peut demander
static int assembler = 0; // Flag : outils obtenus → peut assembler
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t  c_pret = PTHREAD_COND_INITIALIZER;
static pthread_cond_t  c_asm  = PTHREAD_COND_INITIALIZER;
// Thread 1 : signaler que la réflexion est terminée
pthread_mutex_lock(&mutex);
pret = 1;
pthread_cond_signal(&c_pret);
pthread_mutex_unlock(&mutex);
// Thread 2 : attendre le signal de réflexion
pthread_mutex_lock(&mutex);
while (!pret) pthread_cond_wait(&c_pret, &mutex);
pret = 0;
pthread_mutex_unlock(&mutex);

```

4) Cycle de vie d'un bras robotique

Chaque instance de bras robotique suit un cycle d'états piloté par trois threads coopérants via mutex et variables de condition :



Guide d'Exécution sur Kubuntu linux

1) Prérequis

Avant de compiler le projet, s'assurer que GCC et Make sont installés :

```
sudo apt update
sudo apt install -y gcc make
```

```

amine@amine-hp250g8notebookpc:~$ sudo apt update
[sudo: authenticate] Password:
Réception de : 1 http://security.ubuntu.com/ubuntu questing-security InRelease [136 kB]
Atteint : 2 http://archive.ubuntu.com/ubuntu questing InRelease
Atteint : 3 https://brave-browser-apt-release.s3.brave.com stable InRelease
Atteint : 4 https://packages.microsoft.com/repos/code stable InRelease
Réception de : 5 http://archive.ubuntu.com/ubuntu questing-updates InRelease [136 kB]
Réception de : 6 http://security.ubuntu.com/ubuntu questing-security/main amd64 Components [3,160 B]
Réception de : 7 http://security.ubuntu.com/ubuntu questing-security/universe amd64 Components [30.6 kB]
Réception de : 8 http://archive.ubuntu.com/ubuntu questing-backports InRelease [133 kB]
Réception de : 9 http://archive.ubuntu.com/ubuntu questing-updates/main amd64 Components [58.0 kB]
Réception de : 10 http://archive.ubuntu.com/ubuntu questing-updates/universe amd64 Components [60.9 kB]
Réception de : 11 http://archive.ubuntu.com/ubuntu questing-updates/multiverse amd64 Components [944 B]
Réception de : 12 http://archive.ubuntu.com/ubuntu questing-backports/universe amd64 Components [5,728 B]
563 ko réceptionnés en 2s (272 ko/s)
218 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
amine@amine-hp250g8notebookpc:~$

```

```

amine@amine-hp250g8notebookpc:~$ sudo apt install -y gcc make
gcc est déjà la version la plus récente (4:15.2.0-4ubuntu1).
make est déjà la version la plus récente (4.4.1-2).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 218
amine@amine-hp250g8notebookpc:~$

```

2) Compilation et exécution — Partie 1

La Partie 1 se compile et s'exécute avec deux commandes seulement :

```

# 1. Se placer dans le dossier
cd ~/Downloads/projet_SE/ordonnanceur

# 2. Compiler
gcc -o ordo ordo.c

# 3. Exécuter
./ordo

```

Un menu interactif s'affiche avec 4 options :

- Option 1 : Simulation FIFO
- Option 2 : Simulation SJF
- Option 3 : Simulation Round Robin
- Option 4 : Les 3 politiques à la suite (recommandé pour comparer)

```

> ordonnanceur : ordo — Konsole
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 218
amine@amine-hp250g8notebookpc:~$ cd ~/Downloads/projet_SE/ordonnanceur
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/ordonnanceur$ gcc -o ordo ordo.c
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/ordonnanceur$ ./ordo

SIMULATION ORDONNANCEUR DE PROCESSUS
Projet SE - ENSEM

Processus chargés :



| Nom | Arrivée | Burst CPU |
|-----|---------|-----------|
| P1  | 0       | 5         |
| P2  | 1       | 3         |
| P3  | 2       | 8         |
| P4  | 3       | 2         |



Choisir une politique :
1 → FIFO (Premier arrivé, premier servi)
2 → SJF (Plus court en premier)
3 → Round Robin (Quantum = 3)
4 → Toutes les politiques

Votre choix [1-4] : 

```

```

> ordonnanceur : bash — Konsole

Politique : FIFO - Premier arrivé, premier servi||



| t  | Processus | Action                 |
|----|-----------|------------------------|
| 0  | P1        | ➤ Arrivée dans la file |
| 0  | P1        | ➤ CPU [ ] 1/5 restant  |
| 1  | P2        | ➤ Arrivée dans la file |
| 1  | P2        | ➤ CPU [ ] 1/3 restant  |
| 2  | P3        | ➤ Arrivée dans la file |
| 2  | P3        | ➤ CPU [ ] 1/8 restant  |
| 3  | P4        | ➤ Arrivée dans la file |
| 3  | P4        | ➤ CPU [ ] 1/2 restant  |
| 4  | [idle]    | zzz CPU libre...       |
| 5  | [idle]    | zzz CPU libre...       |
| 6  | [idle]    | zzz CPU libre...       |
| 7  | [idle]    | zzz CPU libre...       |
| 8  | [idle]    | zzz CPU libre...       |
| 9  | [idle]    | zzz CPU libre...       |
| 10 | [idle]    | zzz CPU libre...       |
| 11 | [idle]    | zzz CPU libre...       |
| 12 | [idle]    | zzz CPU libre...       |
| 13 | [idle]    | zzz CPU libre...       |
| 14 | [idle]    | zzz CPU libre...       |
| 15 | [idle]    | zzz CPU libre...       |
| 16 | [idle]    | zzz CPU libre...       |


```

```

189 [idle] zzz CPU libre...
190 [idle] zzz CPU libre...
191 [idle] zzz CPU libre...
192 [idle] zzz CPU libre...
193 [idle] zzz CPU libre...
194 [idle] zzz CPU libre...
195 [idle] zzz CPU libre...
196 [idle] zzz CPU libre...
197 [idle] zzz CPU libre...
198 [idle] zzz CPU libre...
199 [idle] zzz CPU libre...

```

Proc	Arrivée	Burst	Attente	Retour
P1	0	5	0	-1
P2	1	3	0	-2
P3	2	8	0	-3
P4	3	2	0	-4
Moyenne			0.0	-2.5

```

Simulation terminée.
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/ordonnanceur$

```

3) Compilation et exécution — Partie 2

La Partie 2 nécessite deux terminaux ouverts simultanément. Le serveur doit être lancé en premier.

a. Compiler les deux programmes

```

# Compiler le serveur
cd ~/Downloads/projet_SE/entrepot/serveur
gcc -o serveur serveur.c -lpthread

# Compiler le bras
cd ~/Downloads/projet_SE/entrepot/client
gcc -o bras bras.c -lpthread

```

```

amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/ordonnanceur$ cd ~/Downloads/projet_SE/entrepot/serveur
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/serveur$ gcc -o serveur serveur.c -lpthread
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/serveur$ cd ~/Downloads/projet_SE/entrepot/client
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/client$ gcc -o bras bras.c -lpthread
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/client$

```

b. Lancement (Terminal 1 Serveur) en attente de bras

```

cd ~/Downloads/projet_SE/entrepot/serveur
./serveur

```

```

amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/client$ cd ~/Downloads/projet_SE/entrepot/serveur
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/serveur$ ./serveur

```

```

SERVEUR GESTIONNAIRE D'OUTILS
Entrepôt Intelligent

```

```

✓ 5 outils initialisés
✓ Port : 9090
✓ Max bras : 8

```

```

État des outils :  00:libre  01:libre  02:libre  03:libre  04:libre

```

```

[02:43:26]  Serveur démarré — en attente de bras...

```


C. Lancement (Terminal 2 — Bras robotique)

```
cd ~/Downloads/projet_SE/entrepot/client
```

```
./bras 1          # Lancer le bras numéro 1
```

```
# Dans un 3ème terminal, lancer d'autres bras :
```

```
./bras 2          # Bras numéro 2
```

```
./bras 3          # Bras numéro 3
```

```
amine@amine-hp250g8notebookpc:~$ cd ~/Downloads/projet_SE/entrepot/client
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/client$ ./bras 1
```

BRAS ROBOTIQUE N° 1
 3 threads : réflexion/demande/assemblage

```
[02:44:27] 🔄 Bras 1 → Connecté au serveur !
[02:44:27] 🕒 Bras 1 → En réflexion pendant 3 sec...
[02:44:30] 🕒 Bras 1 → En réflexion pendant 2 sec...
[02:44:30] 📦 Bras 1 → Demande outils 00 et 03 au serveur...
[02:44:32] 🕒 Bras 1 → En réflexion pendant 1 sec...
[02:44:33] 🕒 Bras 1 → En réflexion pendant 3 sec...
[02:44:36] 🕒 Bras 1 → En réflexion pendant 1 sec...
[02:44:37] 🕒 Bras 1 → En réflexion pendant 2 sec...
[02:44:39] 🕒 Bras 1 → En réflexion pendant 2 sec...
[02:44:41] 🕒 Bras 1 → En réflexion pendant 3 sec...
[02:44:44] 🕒 Bras 1 → En réflexion pendant 3 sec...
[02:44:47] 🕒 Bras 1 → En réflexion pendant 1 sec...
[02:44:48] 🕒 Bras 1 → En réflexion pendant 2 sec...
[02:44:50] 🕒 Bras 1 → En réflexion pendant 3 sec...
```

```
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/client$ ./bras 2
```

BRAS ROBOTIQUE N° 2
 3 threads : réflexion/demande/assemblage

```
[02:46:21] 🔄 Bras 2 → Connecté au serveur !
[02:46:21] 🕒 Bras 2 → En réflexion pendant 2 sec...
[02:46:23] 🕒 Bras 2 → En réflexion pendant 1 sec...
[02:46:23] 📦 Bras 2 → Demande outils 01 et 04 au serveur...
[02:46:24] 🕒 Bras 2 → En réflexion pendant 2 sec...
[02:46:26] 🕒 Bras 2 → En réflexion pendant 2 sec...
[02:46:28] 🕒 Bras 2 → En réflexion pendant 1 sec...
[02:46:29] 🕒 Bras 2 → En réflexion pendant 2 sec...
[02:46:31] 🕒 Bras 2 → En réflexion pendant 3 sec...
[02:46:34] 🕒 Bras 2 → En réflexion pendant 1 sec...
[02:46:35] 🕒 Bras 2 → En réflexion pendant 1 sec...
[02:46:36] 🕒 Bras 2 → En réflexion pendant 3 sec...
[02:46:39] 🕒 Bras 2 → En réflexion pendant 3 sec...
[02:46:42] 🕒 Bras 2 → En réflexion pendant 1 sec...
[02:46:43] 🕒 Bras 2 → En réflexion pendant 1 sec...
[02:46:44] 🕒 Bras 2 → En réflexion pendant 1 sec...
[02:46:45] 🕒 Bras 2 → En réflexion pendant 3 sec...
[02:46:48] 🕒 Bras 2 → En réflexion pendant 3 sec...
```

```

client : bras — Konsole
amine@amine-hp250g8notebookpc:~$ cd ~/Downloads/projet_SE/entrepot/client
amine@amine-hp250g8notebookpc:~/Downloads/projet_SE/entrepot/client$ ./bras 3

BRAS ROBOTIQUE N° 3
3 threads : réflexion/demande/assemblage

[02:48:12] Bras 3 → Connecté au serveur !
[02:48:12] Bras 3 → En réflexion pendant 1 sec...
[02:48:13] Bras 3 → En réflexion pendant 1 sec...
[02:48:13] Bras 3 → Demande outils 00 et 02 au serveur...
[02:48:14] Bras 3 → En réflexion pendant 2 sec...
[02:48:16] Bras 3 → En réflexion pendant 1 sec...
[02:48:17] Bras 3 → En réflexion pendant 3 sec...
[02:48:20] Bras 3 → En réflexion pendant 2 sec...
[02:48:22] Bras 3 → En réflexion pendant 1 sec...
[02:48:23] Bras 3 → En réflexion pendant 3 sec...
[02:48:26] Bras 3 → En réflexion pendant 2 sec...
[02:48:28] Bras 3 → En réflexion pendant 1 sec...
[02:48:29] Bras 3 → En réflexion pendant 1 sec...
[02:48:30] Bras 3 → En réflexion pendant 2 sec...

```

4) Lecture de l'affichage

a. Affichage de l'ordonnanceur

L'ordonnanceur affiche un tableau tick par tick avec une barre de progression pour chaque processus :

```

ordonnanceur : bash — Konsole

Politique : SJF — Plus court en premier ||

t  Processus  Action
0  P1          ► Arrivée dans la file
0  P1          ► CPU [███] 1/5 restant
1  P2          ► Arrivée dans la file
1  P2          ► CPU [██] 1/3 restant
2  P3          ► Arrivée dans la file
2  P3          ► CPU [██] 1/8 restant
3  P4          ► Arrivée dans la file
3  P4          ► CPU [██] 1/2 restant
4  [idle]      zzz CPU libre...
5  [idle]      zzz CPU libre...
6  [idle]      zzz CPU libre...
7  [idle]      zzz CPU libre...
8  [idle]      zzz CPU libre...
9  [idle]      zzz CPU libre...
10 [idle]      zzz CPU libre...
11 [idle]      zzz CPU libre...
12 [idle]      zzz CPU libre...
13 [idle]      zzz CPU libre...
14 [idle]      zzz CPU libre...
15 [idle]      zzz CPU libre...

```

186	[idle]	zzz CPU libre...
187	[idle]	zzz CPU libre...
188	[idle]	zzz CPU libre...
189	[idle]	zzz CPU libre...
190	[idle]	zzz CPU libre...
191	[idle]	zzz CPU libre...
192	[idle]	zzz CPU libre...
193	[idle]	zzz CPU libre...
194	[idle]	zzz CPU libre...
195	[idle]	zzz CPU libre...
196	[idle]	zzz CPU libre...
197	[idle]	zzz CPU libre...
198	[idle]	zzz CPU libre...
199	[idle]	zzz CPU libre...

STATISTIQUES				
Proc	Arrivée	Burst	Attente	Retour
P1	0	5	0	-1
P2	1	3	0	-2
P3	2	8	0	-3
P4	3	2	0	-4
Moyenne			0.0	-2.5

b. Serveur après la connexion du bras 2

Le serveur affiche l'état de chaque outil en temps réel après chaque acquisition ou libération :

serveur : serveur — Konsole				
✓ Max bras : 8				
État des outils : 🗑️ 00:libre 🗑️ 01:libre 🗑️ 02:libre 🗑️ 03:libre 🗑️ 04:libre				
[02:43:26] 🚧 Serveur démarré — en attente de bras...				
[02:44:27] 🚧 Bras 1 connecté (socket 4)				
[02:44:30] 🚧 Bras 1 demande outils 00 et 03				
[02:44:30] ✓ Bras 1 obtient outil 00				
État des outils : 🔒 00:Bras1 🗑️ 01:libre 🗑️ 02:libre 🗑️ 03:libre 🗑️ 04:libre				
[02:44:30] ✓ Bras 1 obtient outil 03				
État des outils : 🔒 00:Bras1 🗑️ 01:libre 🗑️ 02:libre 🔒 03:Bras1 🗑️ 04:libre				
[02:46:21] 🚧 Bras 2 connecté (socket 5)				
[02:46:23] 🚧 Bras 2 demande outils 01 et 04				
[02:46:23] ✓ Bras 2 obtient outil 01				
État des outils : 🔒 00:Bras1 🔒 01:Bras2 🗑️ 02:libre 🔒 03:Bras1 🗑️ 04:libre				
[02:46:23] ✓ Bras 2 obtient outil 04				
État des outils : 🔒 00:Bras1 🔒 01:Bras2 🗑️ 02:libre 🔒 03:Bras1 🔒 04:Bras2				
[02:46:49] 🚧 Bras 2 déconnecté				

Conclusion

Ce projet a permis d'implémenter et de comprendre les principaux mécanismes d'un système d'exploitation moderne à travers deux applications concrètes en langage C.

La Partie 1 illustre comment un ordonnanceur OS gère les processus en attribuant le CPU selon différentes politiques. La comparaison entre FIFO, SJF et Round Robin permet de constater concrètement l'impact de l'algorithme sur les temps d'attente et de retour.

La Partie 2 simule un système concurrent réel avec la problématique classique des ressources partagées. L'utilisation de mutex, de variables de condition et de la stratégie d'ordre d'acquisition anti-deadlock démontre comment éviter les situations de blocage dans un environnement multi-thread.

Concept appris	Mise en pratique
Ordonnancement CPU	3 algorithmes : FIFO, SJF, Round Robin
Threads POSIX	pthread_create, pthread_join
Synchronisation	mutex + pthread_cond_wait/signal
Prévention des deadlocks	Ordre croissant d'acquisition
Sockets TCP	Client/serveur en C avec accept/recv/send
Concurrence	Plusieurs bras accédant aux mêmes outils



Manar Bouamar

Amine Andame

Fin du rapport